

COURSE CODE/TITLE:	CPE 010	SECTION:	CPE21S2
DATE PERFORMED:	July 24, 2026	DATE SUBMITTED:	July 29, 2026
NAME:	Johrel Louie T. Silao	INSTRUCTOR:	Engr. Jimlord M. Quejado

ACTIVITY NO. <u>3.1</u>	Linked Lists
1. PROCEDURE OUTPUT	

Screenshot	<pre> 1 #include<iostream> 2 #include<utility> 3 4 class Node{ 5 public: 6 char data; 7 Node *next; 8 }; 9 10 int main(){ 11 //step 1 12 Node *head = NULL; 13 Node *second = NULL; 14 Node *third = NULL; 15 Node *fourth = NULL; 16 Node *fifth = NULL; 17 Node *last = NULL; 18 //step 2 19 head = new Node; 20 second = new Node; 21 third = new Node; 22 fourth = new Node; 23 fifth = new Node; 24 last = new Node; 25 //step 3 26 head->data = 'C'; 27 head->next = second; 28 second->data = 'P'; 29 second->next = third; 30 third->data = 'E'; 31 third->next = fourth; 32 fourth->data = '0'; 33 fourth->next = fifth; 34 fifth->data = '1'; 35 fifth->next = last; 36 //step 4 37 last->data = '0'; 38 last->next = nullptr; 39 </pre>
-------------------	---

	<pre> * Executing task: C/C++: clang++ build active file Starting build... /usr/bin/clang++ -std=gnu++14 -fcolor-diagnostics -fansi-escape-c 0_SILA0/H0A_3.1_SILA0/simpleImplementation Build finished successfully. * Terminal will be reused by tasks, press any key to close it. </pre>
Discussion	This code structure functions correctly, however, it is suffering from memory leaks. It allocates node memory on the heap using "new: without the delete" leaving the unallocated memory unsupervised.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	<pre> template <typename T> void ListTraversal(SingleList<T>* head){ if(head == nullptr){ std::cout<<"The list is empty."; return; } while (head != nullptr){ // Print data of N std::cout << head->data; if(head->next != nullptr){ std::cout << " -> "; } // Go to the next node head = head->next; } std::cout << std::endl; } </pre>

Insertion at head	<pre> template <typename T> void sll_insert_head(T newData, SingleList<T>*& currentHead){ // 1. Allocate memory for the new node SingleList<T>* newNode = new SingleList<T>; // 2. Put our data into the new node newNode->data = newData; // 3. Set the next of the new node to point to the previous head newNode->next = *currentHead; // 4. Reset head to point to the new node *currentHead = newNode; } </pre>
Insertion at any part of the list	<pre> template <typename T> // 1. Check if it is the head node(previous node is null) void sll_general_insert(T newData, SingleList<T>*& prevNode){ // 2. If null, print "Previous node cannot be nullptr" and return if(prevNode == nullptr){ std::cout <<"Previous node cannot be nullptr" << std::endl; return; } // 3. Allocate a new node SingleList<T>* newNode = new SingleList<T>; // 4. Store data in the new node newNode->data = newData; // 5. Point to the new node to the node previous node was pointing to newNode->next = prevNode->next; // 6. Point previous node to the new node prevNode-> next = newNode; } </pre>

Insertion at the end	<pre> template <typename T> void sll_insert_end(T newData, SingleList<T>** head){ // 1. Allocate new node SingleList<T>* newNode = new SingleList<T>; // 2. Dereference to the head node SingleList<T>* currentNode = *head; // 3. Store data in new node newNode->data = newData; // 4. Point to null newNode->next = nullptr; // 5. Traverse the list until pointed to null while(currentNode->next != nullptr){ currentNode = currentNode->next; } // 6. Point the next of the current node to the new node currentNode->next = newNode; } </pre>
Deletion of a node	<pre> template <typename T> void sll_Delete_Node(T findData, SingleList<T>** head){ // If there is nothing to delete if(*head == nullptr) return; // SingleList<T>* currNode = *head; SingleList<T>* prevNode = nullptr; // Searching the node: 1. the node does not exist + we found the data while(currNode != nullptr && currNode->data != findData){ prevNode = currNode; currNode = currNode->next; } // If data not found currNode == nullptr in the while loop: if(currNode == nullptr){ std::cout<<"The data is not found \n"<<std::endl; } // If data was found if(prevNode == nullptr){ *head = currNode->next; }else{ prevNode->next = currNode->next; } delete currNode; } </pre>

Table 3-2. Code for the List Operations

a.	Source Code	<pre> template <typename T> void ListTraversal(SingleList<T>* head){ if(head == nullptr){ std::cout<<"The list is empty."; return; } while (head != nullptr){ // Print data of N std::cout << head->data; if(head->next != nullptr){ std::cout << " -> "; } // Go to the next node head = head->next; } std::cout << std::endl; } </pre>
	Console	<pre> Testing of Traversal: J -> 0 -> L -> 0 </pre>

b.	Source Code	<pre>template <typename T> void sll_insert_head(T newData, SingleList<T>*& currentHead){ // 1. Allocate memory for the new node SingleList<T>*& newNode = new SingleList<T>; // 2. Put our data into the new node newNode->data = newData; // 3. Set the next of the new node to point to the previous head newNode->next = *currentHead; // 4. Reset head to point to the new node *currentHead = newNode; }</pre>
	Console	<pre>Testing of sll_insert_head: Q -> J -> 0 -> L -> 0</pre>
c.	Source Code	<pre>template <typename T> // 1. Check if it is the head node(previous node is null) void sll_general_insert(T newData, SingleList<T>*& prevNode){ // 2. If null, print "Previous node cannot be nullptr" and return if(prevNode == nullptr){ std::cout <<"Previous node cannot be nullptr" << std::endl; return; } // 3. Allocate a new node SingleList<T>*& newNode = new SingleList<T>; // 4. Store data in the new node newNode->data = newData; // 5. Point to the new node to the node previous node was pointing to newNode->next = prevNode->next; // 6. Point previous node to the new node prevNode-> next = newNode; }</pre>
	Console	<pre>Testing of sll_general_insert: Q -> J -> 0 -> X -> L -> 0</pre>

d.	Source Code	<pre> template <typename T> void sll_insert_end(T newData, SingleList<T>*& head){ // 1. Allocate new node SingleList<T>* newNode = new SingleList<T>; // 2. Dereference to the head node SingleList<T>* currentNode = *head; // 3. Store data in new node newNode->data = newData; // 4. Point to null newNode->next = nullptr; // 5. Traverse the list until pointed to null while(currentNode->next != nullptr){ currentNode = currentNode->next; } // 6. Point the next of the current node to the new node currentNode->next = newNode; } </pre>
	Console	<pre> Testing of sll_insert_end: Q -> J -> O -> X -> L -> O -> U </pre>

e.	Source Code	<pre> template <typename T> void sll_Delete_Node(T findData, SingleList<T>*& head){ // If there is nothing to delete if(*head == nullptr) return; // SingleList<T>* currNode = *head; SingleList<T>* prevNode = nullptr; // Searching the node: 1. the node does not exist + we found the data while(currNode != nullptr && currNode->data != findData){ prevNode = currNode; currNode = currNode->next; } // If data not found currNode == nullptr in the while loop: if(currNode == nullptr){ std::cout<<"The data is not found \n"<<std::endl; } // If data was found if(prevNode == nullptr){ *head = currNode->next; }else{ prevNode->next = currNode->next; } delete currNode; } </pre>
	Console	<pre> Testing of deleting the node: Q -> J -> 0 -> X -> L -> 0 J -> 0 -> X -> L -> 0 J -> 0 -> L -> 0 </pre>

f.	Source Code	<pre>template <typename T> void sll_Deleting_List(SingleList<T> **head){ SingleList<T>* current = *head; while(current != nullptr){ SingleList<T>* temp = current; current = current->next; delete temp; } *head = nullptr; } #endif</pre>
	Console	<pre>Deleting all nodes: The list is empty. %</pre>

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshot(s)	Analysis
<pre>template <typename T> class DoubleList { public: T data; DoubleList<T>* prev = nullptr; DoubleList<T>* next = nullptr; }; // Creating a Node template <typename T> DoubleList<T> *CreateNewNode(T newData) { // Dynamically allocate a new memory for the node DoubleList<T> *newNode = new DoubleList<T>; // Store the data in the node newNode->data = newData; // Point next and prev to null newNode->next = nullptr; newNode->prev = nullptr; // Return the address of the newly created node return newNode; }</pre>	This code snippet shows the template class DoubleList containing a data field and two pointers, the “prev” and “next.” They are initialized to “nullptr” to represent that they are in a double linked list. The function “CreateNewNode” allocates the memory on the heap for the new “DoubleList.”

```
template <typename T>
void dllTraverse(DoubleList<T>* currentNode) {

    DoubleList<T> *tail;
    // Check if the list is empty
    if (currentNode == nullptr) {
        std::cout << "The list is empty." << std::endl;
        return;
    }

    // Move forward
    while (currentNode != nullptr) {
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }

    // Adding a next line
    std::cout << std::endl;

    // Move backward
    while (tail != nullptr) {
        std::cout << tail->data << "";
        tail = tail->prev;
    }
    std::cout << std::endl;
}
```

The “dllTraverse” function checks if the given list node is a “nullptr” which prints an empty message. The forward pass iterates through every node using “next” pointers to display the data. Then the “tail” will iterate backwards using “prev” pointers, which will display the elements in a reversed order.

```
// Inserting at the head of the list
template <typename T>
void dllInsertHead(T newData, DoubleList<T> ** currentHead) {
    // Creating a new node
    DoubleList<T>* newNode = CreateNewNode(newData);

    // New node should point to the current head
    newNode->next = *currentHead;

    // Current Head should point back to the newNode
    (*currentHead) ->prev = newNode;

    // Update the pointer head
    *currentHead = newNode;
}
```

In this snippet, the “dllInsertHead” inserts a new data element at the beginning of an existing double list. It starts by applying the “CreateNew” to construct a new node in the “newData” and the new node points to the head, and the original head’s pointer is back to the newNode. Lastly, the dereferenced pointer is updated to point to the newNode, making it the new first element in the list.

```
template <typename T>
void dllInsertTail(T newData, DoubleList<T>* currentHead) {
    // Create a new node
    DoubleList<T>* newNode = CreateNewNode(newData);

    // Traverse until we reach the last node
    while(currentHead->next != nullptr) {
        currentHead = currentHead->next;
    }

    // Connect the last node to the new node
    currentHead->next = newNode;

    // Connect the new node back to the last node
    newNode->prev = currentHead;
}
```

This function appends a new node at the tail end of the double linked list. It illustrates a new node using the “CreateNewNode” and traverses forward using the “while” loop, reaching the last node the “next;” == nullptr.” At the end, it links the “currentHead -> next” to “newNode” and it will set the “newNode ->prev” back to the “currentHead.”

```
// General Insert
template <typename T>

void dllGeneralInsert (T newData, DoubleList<T>* prevNode) {
    if (prevNode == nullptr) {
        std::cout << "Previous node cannot be nullptr!" << std::endl;
        return;
    }
    DoubleList<T>* newNode = new DoubleList<T>;

    newNode->data = newData;

    newNode->next = prevNode->next;

    newNode->prev = prevNode;

    if (newNode->next != nullptr) {
        newNode->next->prev = newNode;
    }

    prevNode->next = newNode;
}
```

The “dllGeneralInsert” inserts a new node after a specified target node in the list. It also performs an error checking to ensure that the “prevNode” is not a “nullptr” before allocating and copying the “newData” in it. Additionally, it will link the new node by reassigning both pointers.

```
// Delete a node
template <typename T>

void dllDeleteNode (DoubleList<T>* head, T findData) {
    DoubleList<T>* currNode = head;

    while (currNode != nullptr && currNode->data != findData) {
        currNode = currNode->next;
    }

    if (currNode == nullptr) {
        std::cout << "The data is not found" << std::endl;
        return;
    }

    if (head == currNode) {
        head = currNode->next;
    }

    if (currNode->next != nullptr) {
        currNode->next->prev = currNode->prev;
    }

    if (currNode->prev != nullptr) {
        currNode->prev->next = currNode->next;
    }

    delete currNode;
}

#endif
```

The function “dllDeleteNode” finds a target value and unlinks its corresponding node from the double linked list. It will traverse the list until it ends up in the nullptr. Whenever a node is the head of the list, the pointer is updated before calling to free the memory.

Table 3-4. Modified Operations for Doubly Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++

Problem Title: Implementing a Song Playlist using Linked List

Source: Packt Publishing

Problem Description:

In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

We'll take the use case of a music player. It should have following functions supported:

1. Create a playlist using multiple songs.
2. Add songs to the playlist.
3. Remove a song from the playlist.
4. Play songs in a loop (for this activity, we will print all the songs once).

Here are the steps to solve the problem:

1. Design the basic structure that supports circular data representation.
2. After that, implement the insert and delete functions in the structure.
3. Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

```
1  #include <iostream>
2  #include <string>
3  #include "playlist_ll.h"
4
5  void displayMenu() {
6      std::cout << "\n===== " << std::endl;
7      std::cout << "      MUSIC PLAYLIST MENU      " << std::endl;
8      std::cout << "===== " << std::endl;
9      std::cout << "1. Insert Song at Beginning" << std::endl;
10     std::cout << "2. Insert Song at End" << std::endl;
11     std::cout << "3. Next Song" << std::endl;
12     std::cout << "4. Previous Song" << std::endl;
13     std::cout << "5. Display Currently Playing Song" << std::endl;
14     std::cout << "6. Play All Songs (Loop Traversal)" << std::endl;
15     std::cout << "7. Remove a Song" << std::endl;
16     std::cout << "8. Exit" << std::endl;
17     std::cout << "===== " << std::endl;
18     std::cout << "Enter choice: ";
19 }
```

Figure 1.1: Code snippet of mainPlaylist.cpp

Analysis:

- Displays a menu interface that handles a music playlist using a template circular double linked list. It runs on a “while” loop with a “switch-case,” which allows the user to add, remove, navigate, display and play all songs from memory.

```

21  int main() {
22      circList<std::string>* playlistHead = nullptr;
23      circList<std::string>* currentSong = nullptr;
24
25      int choice = 0;
26      std::string songTitle;
27
28      while (true) {
29          displayMenu();
30          if (!(std::cin >> choice)) {
31              std::cin.clear();
32              std::cin.ignore(10000, '\n');
33              continue;
34          }
35          std::cin.ignore(); // Clear newline left in buffer
36
37          switch (choice) {
38              case 1:
39                  std::cout << "Enter song title to insert at start: ";
40                  std::getline(std::cin, songTitle);
41                  insertHead(songTitle, &playlistHead);
42
43                  // Set currentSong to head if this is the first song inserted
44                  if (currentSong == nullptr) {
45                      currentSong = playlistHead;
46                  }
47                  std::cout << "\"" << songTitle << "\" added to the start.\n";
48                  break;
49
50              case 2:
51                  std::cout << "Enter song title to insert at end: ";
52                  std::getline(std::cin, songTitle);
53
54                  if (playlistHead == nullptr) {
55                      insertHead(songTitle, &playlistHead);
56                      currentSong = playlistHead;
57                  } else {
58                      insertEnd(songTitle, playlistHead);
59                  }
60                  std::cout << "\"" << songTitle << "\" added to the end.\n";
61                  break;

```

Figure 1.2: Code snippet of mainPlaylist.cpp

Analysis

- Defines the display menu function with 8 choices to the user, consisting of the main foundation of the playlist including the direct navigation. Case 1 and 2 handles by taking the string inputs and calls the insertHead to link new songs into the circular sequence and automatically initializes the pointer to the first song.

```

63     case 3:
64         if (currentSong == nullptr) {
65             std::cout << "Playlist is empty." << std::endl;
66         } else {
67             nextSong(&currentSong);
68             std::cout << "Now playing: " << currentSong->data << "\n";
69         }
70         break;
71
72     case 4:
73         if (currentSong == nullptr) {
74             std::cout << "Playlist is empty." << std::endl;
75         } else {
76             prevSong(&currentSong);
77             std::cout << "Now playing: " << currentSong->data << "\n";
78         }
79         break;
80
81     case 5:
82         std::cout << "Currently Playing: ";
83         displayCurrent(currentSong);
84         break;
85
86     case 6:
87         std::cout << "\n--- Playing All Songs --- " << std::endl;
88         playAll(playlistHead);
89         break;
90
91     case 7:
92         if (playlistHead == nullptr) {
93             std::cout << "Playlist is empty." << std::endl;
94             break;
95         }
96         std::cout << "Enter song title to remove: ";
97         std::getline(std::cin, songTitle);
98         deleteNode(songTitle, &playlistHead, &currentSong);
99         break;

```

Figure 1.3: Code snippet of mainPlaylist.cpp

Analysis:

- Case 3, 4 and 5 handles the nextSong and the prevSong to update the currentSong pointer by utilizing the node's circular links, which will help enable the loop of forward and backward without reaching the nullptr. While case 5 handles the displayCurrent to display the current song of the playlist. Case 6 handles the playAll, making it traverse the playlist from head back to itself to display all the songs in the playlist. Case 7 handles the deleteNode to remove the song in the playlist that was added.

```

        case 8:
            std::cout << "Cleaning up memory and exiting..." << std::endl;
            destroyList(&playlistHead);
            return 0;

        default:
            std::cout << "Invalid choice. Please try again." << std::endl;
            break;
    }

    return 0;
}

```

Figure 1.4: Code snippet of mainPlaylist.cpp

Analysis:

- Case 8 handles the destroyList, which deallocates all the dynamically created nodes before exiting the program.

```

=====
                MUSIC PLAYLIST MENU
=====
1. Insert Song at Beginning
2. Insert Song at End
3. Next Song
4. Previous Song
5. Display Currently Playing Song
6. Play All Songs (Loop Traversal)
7. Remove a Song
8. Exit
=====
Enter choice: 1
Enter song title to insert at start: Back to Friends
"Back to Friends" added to the start.

```

Figure 2.1: Output snippet of mainPlaylist.cpp

Analysis:

- The program displays the music playlist menu interface for the user. User inputs choice 1 to insert a song at the beginning of the playlist, which successfully builds a list.

```
=====
MUSIC PLAYLIST MENU
=====
1. Insert Song at Beginning
2. Insert Song at End
3. Next Song
4. Previous Song
5. Display Currently Playing Song
6. Play All Songs (Loop Traversal)
7. Remove a Song
8. Exit
=====
Enter choice: 2
Enter song title to insert at end: Pahina
"Pahina " added to the end.
```

Figure 2.2: Output snippet of mainPlaylist.cpp

Analysis:

- The user inputs choice 2 to enter a song at the end of the playlist.

```
=====
MUSIC PLAYLIST MENU
=====
1. Insert Song at Beginning
2. Insert Song at End
3. Next Song
4. Previous Song
5. Display Currently Playing Song
6. Play All Songs (Loop Traversal)
7. Remove a Song
8. Exit
=====
Enter choice: 3
Now playing: Pahina
```

Figure 2.3: Output snippet of mainPlaylist.cpp

Analysis:

- User inputs choice 3 which successfully loops forward to "Pahina."

```
=====
MUSIC PLAYLIST MENU
=====
1. Insert Song at Beginning
2. Insert Song at End
3. Next Song
4. Previous Song
5. Display Currently Playing Song
6. Play All Songs (Loop Traversal)
7. Remove a Song
8. Exit
=====
Enter choice: 4
Now playing: Back to Friends
```

Figure 2.4: Output snippet of mainPlaylist.cpp

Analysis:

- User inputs choice 4 which wraps backward to "Back to Friends."

```
=====
MUSIC PLAYLIST MENU
=====
1. Insert Song at Beginning
2. Insert Song at End
3. Next Song
4. Previous Song
5. Display Currently Playing Song
6. Play All Songs (Loop Traversal)
7. Remove a Song
8. Exit
=====
Enter choice: 5
Currently Playing: Back to Friends
```

Figure 2.5: Output snippet of mainPlaylist.cpp

Analysis:

- User inputs choice 5 to display the current song of the playlist.

```
=====
                        MUSIC PLAYLIST MENU
=====
1. Insert Song at Beginning
2. Insert Song at End
3. Next Song
4. Previous Song
5. Display Currently Playing Song
6. Play All Songs (Loop Traversal)
7. Remove a Song
8. Exit
=====
Enter choice: 6

--- Playing All Songs ---
Back to Friends Pahina
```

Figure 2.6: Output snippet of mainPlaylist.cpp

Analysis:

- User inputs choice 6 to play all the songs in the playlist which displays all the songs that the user previously inputted.

```
=====
                        MUSIC PLAYLIST MENU
=====
1. Insert Song at Beginning
2. Insert Song at End
3. Next Song
4. Previous Song
5. Display Currently Playing Song
6. Play All Songs (Loop Traversal)
7. Remove a Song
8. Exit
=====
Enter choice: 7
Enter song title to remove: Back to Friends
```

Figure 2.7: Output snippet of mainPlaylist.cpp

Analysis:

- User inputs choice 7 which allows the user to target a specific song to remove in the playlist.

```
=====
                        MUSIC PLAYLIST MENU
=====
1. Insert Song at Beginning
2. Insert Song at End
3. Next Song
4. Previous Song
5. Display Currently Playing Song
6. Play All Songs (Loop Traversal)
7. Remove a Song
8. Exit
=====
Enter choice: 8
Cleaning up memory and exiting...
```

Figure 2.8: Output snippet of mainPlaylist.cpp

Analysis:

- User inputs choice 8 to exit the program and clean up the memory.

Conclusion:

In this laboratory activity, I was able to construct a C++ code for a singly and doubly linked list in C++. Also, I was able to solve given problems utilizing linked lists in C++. I learned that the elements in a linked list are not stored at a contiguous location, they are linked using pointers. The benefits of using linked lists in C++ is that they have a dynamic size and an ease of insertion and deletion. Doing this hands-on activity wasn't easy due to its complexity and complicated naming of variables. I encountered a lot of errors and issues during this activity, specifically the proper naming of variables and using the appropriate pointers. Although I finished this activity late, I am determined to finish the activity with what I learned in this activity. I plan to use these mistakes, lessons and errors in the future, so that whenever I encounter these types of problems, I will be familiar to solve the given problem at hand.



3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.